

Service numérique de données territoriales Hauts-de-France

Épreuve E2 – Préparation d'un projet data

Certification Data Engineer | **EL BREK HAMZA** | Mars 2026

Table des matières

1	Note de synthèse – Compréhension du besoin data	2
1.1	Contexte et enjeux	2
1.2	Reformulation du besoin et objectifs SMART	2
1.3	Parties prenantes	2
2	Avant-projet	3
2.1	Méthode et organisation	3
2.2	Planning prévisionnel	3
2.3	Budget prévisionnel	3
3	Topographie des données	3
3.1	Catalogue des sources	3
3.2	Métadonnées clés	4
3.3	Flux de données – Architecture logique	5
3.4	Modèle de données – Tables principales	5
4	Architecture technique	6
4.1	Choix technologiques et justifications	6
4.2	Infrastructure as Code – Terraform	6
4.3	Conformité RGPD et éco-responsabilité	6
5	Bilan opérationnel et financier	7
5.1	Réalisations – Indicateurs opérationnels	7
5.2	Bilan financier	7
6	Retour d'expérience – Problèmes et arbitrages	7
7	Bulletin de veille technique	8
7.1	Qu'est-ce que le format Apache Parquet ?	8
7.2	L'architecture Data Lakehouse	9
7.3	Tendances et recommandations	9

1 Note de synthèse – Compréhension du besoin data

1.1 Contexte et enjeux

L'établissement bancaire opère dans la région **Hauts-de-France** et conduit un plan stratégique à horizon 2030 visant à consolider et étendre son réseau d'agences sur les 5 départements de la région (Nord – 59, Pas-de-Calais – 62, Somme – 80, Oise – 60, Aisne – 02). Pour alimenter ces décisions d'implantation, la Direction Stratégie a exprimé un besoin data structuré autour d'une question centrale : *quels territoires de la région présentent le meilleur potentiel pour l'ouverture de nouvelles agences ?*

L'absence d'un outil centralisé contraignait les analystes à des exports manuels depuis data.gouv.fr, sources d'erreurs et de pertes de temps significatives. Ce projet vise à combler ce manque par un service numérique pérenne, automatisé et sécurisé.

1.2 Reformulation du besoin et objectifs SMART

Besoin reformulé

Concevoir un **pipeline de données automatisé** collectant, transformant et exposant des indicateurs territoriaux (socio-économiques, démographiques, géographiques, immobiliers) pour l'ensemble des communes des Hauts-de-France, accessibles via une API REST sécurisée.

Objectif	Formulation SMART
O1 – Collecte	Automatiser l'extraction depuis ≥ 4 sources hétérogènes (API Géo, CSV INSEE, scraping Meilleurtaux, SIRENE Parquet) couvrant les $\approx 3\,700$ communes des 5 départements HdF avec ≥ 8 indicateurs/commune. Délai : fin mois 2 .
O2 – Stockage	Charger les données dans une base relationnelle Azure SQL (schéma 3NF, $< 5\%$ valeurs manquantes, requêtes $< 2s$). Délai : fin mois 3 .
O3 – Exposition	Déployer une API REST (FastAPI) avec authentification par clé API, disponibilité $\geq 99\%$, consultable par commune et département. Délai : fin mois 4 .

1.3 Parties prenantes

Partie prenante	Rôle	Attentes
Direction Stratégie 2030	Commanditaire	Données fiables pour décisions d'implantation
Analystes métiers	Exploitants	API et exports CSV faciles d'utilisation
DSI / SI métiers	Producteurs	Intégration sans perturbation des SI existants
Équipe Conformité	Contrôle RGPD	Sécurité des accès, registre des traitements
Data Engineer (EL BREK H.)	Maîtrise d'œuvre	Réalisation technique du pipeline

2 Avant-projet

2.1 Méthode et organisation

Le projet est conduit en méthode **agile allégée** avec des sprints de 2 semaines et des points hebdomadaires avec le commanditaire. Les livrables sont versionnés sur GitHub ([haelbrek/Projet-Data-ENG](#)) avec une branche **develop** intégrée dans **main** via Pull Requests.

Rôles et responsabilités :

Rôle	Responsabilités
Data Engineer (EL BREK H.)	Développement des pipelines ETL, modélisation BDD, déploiement API, infrastructure Terraform
Direction Stratégie	Validation des indicateurs métiers, recette fonctionnelle
DSI	Accès aux ressources cloud Azure, revue sécurité
Équipe Conformité	Validation RGPD, registre des traitements

Gestion des risques projet :

Risque	Probabilité	Impact	Mitigation
Source externe indisponible	Moyenne	Élevé	Système de cache local + fallback CSV
Dépassement budget Azure	Faible	Moyen	Alertes Azure Monitor sur les coûts
Évolution codes INSEE	Faible	Moyen	Table de correspondance historique
Demande d'intégration DCP	Moyenne	Élevé	Clause AIPD préalable obligatoire

2.2 Planning prévisionnel

M.	Phase	Tâches principales
1	Cadrage	Entretiens parties prenantes · cartographie des sources · choix d'architecture · provisionnement cloud (Terraform)
2	Extraction	Scripts Python (API Géo, CSV INSEE, scraping) · ETL Databricks SIRENE Parquet · upload ADLS Gen2
3	Transformation & Stockage	Nettoyage/normalisation pandas · modélisation MERISE · chargement Azure SQL Database
4	Exposition & Livraison	Développement API FastAPI · déploiement Azure App Service · tests, documentation, recette

2.3 Budget prévisionnel

Poste	Coût mensuel	Détail
Azure SQL Database (Basic)	≈5€	Base relationnelle, sauvegardes auto 7j
Azure Data Lake Storage Gen2	≈10€	Stockage raw/staging/curated
Azure App Service (F1 Free)	0€	Hébergement API REST
Databricks (cluster spot)	≈15€	ETL SIRENE Parquet (usage ponctuel)
Azure Key Vault	≈2€	Gestion des secrets
Azure Monitor + alertes	≈5€	Surveillance et logs
Total infrastructure	≈37€/mois	Coût total projet (4 mois) : ≈150€
Données sources	0€	Open data (data.gouv.fr, INSEE, API Géo)
Ressources humaines	Projet certif.	1 data engineer (formation)

3 Topographie des données

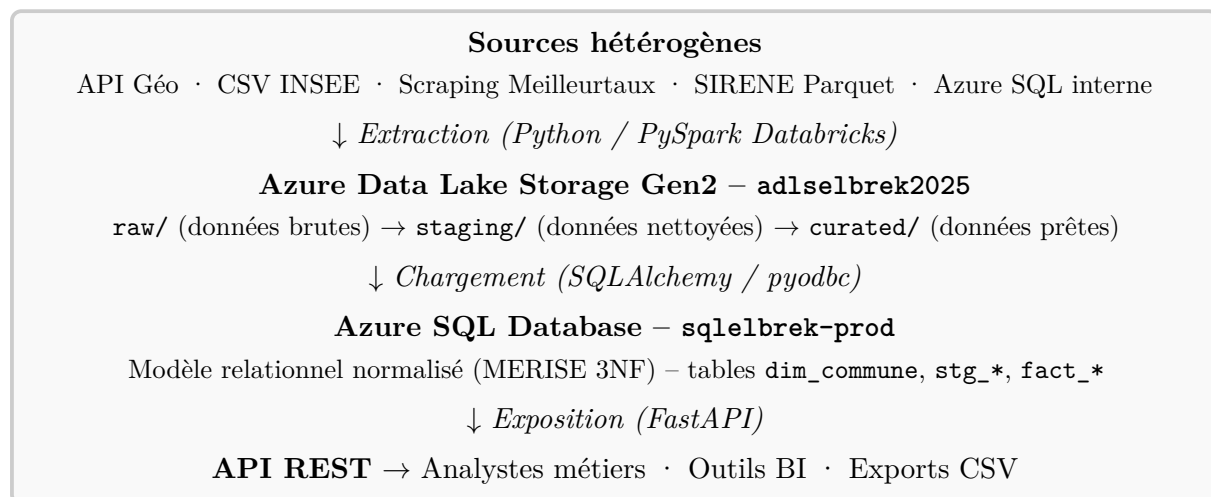
3.1 Catalogue des sources

Source	Fournisseur	Format	Fréquence	Licence	Indicateurs
API Géo	geo.api.gouv.fr	JSON/REST	Temps réel	Open Licence	Code INSEE, population, surface, coordonnées
CSV Démographie	INSEE / data.gouv.fr	CSV	Annuelle	Open Licence	Naissances, décès, fécondité, ménages
CSV Emploi-Chômage	INSEE / data.gouv.fr	CSV	Annuelle	Open Licence	Taux chômage, emplois par secteur
CSV FILOSOFI	INSEE / data.gouv.fr	CSV	Annuelle	Open Licence	Revenus médians, niveau de vie, pauvreté
CSV Logement	INSEE / data.gouv.fr	CSV	Annuelle	Open Licence	Résidences principales, vacance, propriétaires
Scraping Meilleurtaux	Meilleurtaux.com	JSON (AJAX)	Mensuelle	Usage privé	Taux crédit immo par région et durée
SIRENE Parquet	INSEE (Data-bricks)	Parquet	Mensuelle	Open Licence	Établissements actifs HdF, activité NAF, effectifs
Azure SQL (interne)	SI métiers banque	SQL/Parquet	Mensuelle	Interne	Tables territoriales structurées (staging)

3.2 Métadonnées clés

Jeu de données	Identifiant clé	Granularité	Volume	Contrainte qualité
Communes HdF	Code INSEE (5c)	Commune	≈3 700 lignes	0% valeurs manquantes
Démographie	Code INSEE	Commune	≈3 700 lignes	Millésime INSEE ≥2021
FILOSOFI	Code INSEE	Commune	≈3 700 lignes	Seuil secret stat. >500 hab.
Emploi-Chômage	Code INSEE	Commune	≈3 700 lignes	Source officielle Pôle Emploi
SIRENE (filtré HdF)	SIRET (14c)	Établissement	≈400k lignes	Statut actif uniquement
Taux immobiliers	Région + durée	Région	≈30 lignes	Fraîcheur ≤1 mois

3.3 Flux de données – Architecture logique



3.4 Azure Data Lake Storage Gen2 – Organisation du stockage

L'**Azure Data Lake Storage Gen2** (ADLS Gen2) est le socle de stockage central du projet. Il s'agit d'un service de stockage objet hiérarchique d'Azure, conçu pour le traitement analytique de grands volumes de données. Par rapport à un simple stockage Blob, il ajoute la notion de **système de fichiers hiérarchique** (dossiers réels, permissions POSIX), ce qui le rend adapté aux architectures data modernes.

Le compte de stockage **adlsselbrek2025** est organisé en trois zones fonctionnelles (conteneurs), suivant le pattern **Medallion Architecture** :

Zone	Contenu	Rôle et format
raw/	Données brutes sources	Fichiers tels quels à l'ingestion : CSV (INSEE), JSON (API Géo), JSON (scraping), Parquet (SIRENE). Aucune transformation. Conservation à des fins de traçabilité et de rejeu.
staging/	Données intermédiaires	Données nettoyées et validées (types corrects, valeurs manquantes traitées, codes INSEE harmonisés). Format Parquet. Zone de travail avant chargement SQL.
curated/	Données prêtes à l'emploi	Données enrichies et partitionnées prêtes pour l'analyse : curated/etablissements_hdf_actifs/ partitionné par code_departement (02, 59, 60, 62, 80). Format Parquet compressé Snappy.

Accès et sécurité du Data Lake :

- Protocole `abfss://` (Azure Blob File System Secure) pour les accès depuis Databricks (PySpark).
- **RBAC Azure** : rôle *Storage Blob Data Contributor* pour le data engineer, *Storage Blob Data Reader* pour les services consommateurs.
- **Chiffrement** : chiffrement au repos (AES-256) activé par défaut sur Azure ; chiffrement en transit (TLS 1.2).
- **Soft delete** activé sur les conteneurs (rétention 7 jours) pour la récupération accidentelle.
- Accès depuis les scripts Python via `azure-storage-file-datalake` SDK avec `DefaultAzureCredential`.

Exemple de structure des chemins dans le Data Lake :

Listing 1 – Organisation des fichiers dans l'ADLS Gen2

```

1 adlsselbrek2025/
2   raw/
3     csv/           -> communes_hdf.csv, filosofi_2021.csv, ...
4     geo/           -> api_geo_communes.json
5     scraping/      -> taux_immo_meilleurtaux.json
6     Parquet/       -> StockEtablissement_utf8.parquet (30M lignes)
7     sql-bis/       -> dim_commune.parquet, stg_population.parquet, ...
8   curated/
9     etablisements_hdf_actifs/
10      code_departement=02/ -> part-00000.snappy.parquet
11      code_departement=59/ -> part-00000.snappy.parquet
12      code_departement=60/ -> part-00000.snappy.parquet
13      code_departement=62/ -> part-00000.snappy.parquet
14      code_departement=80/ -> part-00000.snappy.parquet

```

3.5 Modèle de données – Tables principales

Table	Description et colonnes clés
dim_commune	Référentiel géographique : <code>code_insee</code> , <code>nom</code> , <code>code_dept</code> , <code>population</code> , <code>surface_km2</code> , <code>latitude</code> , <code>longitude</code>
stg_population	Naissances, décès, solde naturel, solde migratoire par commune et par an
stg_filosofi	Revenus médians, taux de pauvreté, Gini par commune
stg_emploi_chomage	Taux de chômage, emplois par secteur (public/privé)
stg_logement	Résidences principales/secondaires, taux de vacance
stg_creation_entreprises	Créations d'entreprises par commune et secteur NAF
curated.etablissements	Établissements SIRENE actifs HdF partitionnés par département
dim_taux_immo	Taux crédit immobilier par région et durée (source Meilleurtaux)

4 Architecture technique

4.1 Choix technologiques et justifications

Composant	Technologie	Justification
Extraction	Python 3.10+	Écosystème riche (requests, pandas, beautifulsoup4) ; scripts reproductibles et versionnés
Big Data ETL	PySpark / Databricks	Traitement du fichier SIRENE (>30M lignes) impossible en pandas ; cluster spot pour minimiser les coûts
Stockage brut	ADLS Gen2	Stockage hiérarchique (raw/staging/curated), format Parquet colonnaire, intégration native Azure
Base finale	Azure SQL Database	SGBDR robuste, jointures SQL optimisées, sauvegardes automatiques, compatible SQLAlchemy
Infrastructure	Terraform	Provisionnement reproductible et versionné de toute l'infra Azure (IaC)
Exposition	FastAPI	Performance élevée, documentation Swagger auto-générée, authentification par clé API
Secrets	Azure Key Vault	Aucune credential en clair dans le code ; accès RBAC
Orchestration	GitHub Actions	CI/CD intégré, planification des pipelines (cron)

4.2 Infrastructure as Code – Terraform

L'ensemble de l'infrastructure Azure est provisionnée via **Terraform**, garantissant la reproductibilité, le versionnement et la destruction/recréation rapide de l'environnement. Les ressources

déployées incluent :

- **Groupe de ressources rg-data-hdf** (région *France Central*)
- **ADLS Gen2** adlsselbrek2025 avec conteneurs **raw** et **curated**
- **Azure SQL Server** sqlselbrek-prod + Database projet_data_eng (tier Basic, 5 DTU)
- **Azure Key Vault** pour les secrets (chaînes de connexion, clés API)
- **Workspace Databricks** (Standard tier) pour les traitements PySpark
- **Azure Monitor** avec alertes sur les transactions de stockage et la disponibilité de l'API
- **Azure App Service Plan** (F1 Free) pour l'hébergement de l'API FastAPI

L'infrastructure est détruite et recrée à la demande en quelques minutes via **terraform apply**, ce qui garantit une parfaite reproductibilité de l'environnement entre les phases de développement, de test et de production.

4.3 Pipeline ETL – Extrait de code

Listing 2 – Extrait – Filtrage SIRENE Hauts-de-France (PySpark / Databricks)

```

1 # Lecture du fichier SIRENE brut depuis la zone raw/
2 df_raw = spark.read.parquet(
3     "abfss://raw@adlsselbrek2025.dfs.core.windows.net/Parquet/
4     StockEtablissement_utf8.parquet")
5
6 # Filtre : établissements actifs en Hauts-de-France (02/59/60/62/80)
7 df = df_raw.filter(col("etatAdministratifEtablissement") == "A") \
8     .filter(substring(col("codeCommuneEtablissement"),1,2)
9         .isin(["02","59","60","62","80"]))
10
11 # Enrichissement : ajout departement et region
12 df = df.withColumn("code_departement",
13     substring(col("codeCommuneEtablissement"),1,2)) \
14     .withColumn("region", lit("Hauts-de-France"))
15
16 # Sauvegarde partitionnee dans la zone curated/
17 df.write.mode("overwrite").partitionBy("code_departement") \
18     .parquet("abfss://curated@adlsselbrek2025.dfs.core.windows.net
19     /etablisements_hdf_actifs/")

```

4.4 Conformité RGPD et éco-responsabilité

RGPD :

- Données publiques agrégées à la maille commune (non personnelles en v1).
- Logs API : IP hashées, rétention 90 jours.
- Région Azure *France Central* imposée – résidence des données en UE.
- Registre des traitements créé (Art. 30) ; AIPD planifiée pour l'intégration CRM (v2).
- *Privacy by design* : minimisation des données collectées, accès RBAC.

Éco-responsabilité :

- Databricks en cluster **spot** (instances interruptibles) : réduction des coûts et empreinte carbone du traitement SIRENE.
- Format **Parquet** : compression $\times 5$ à $\times 10$ vs CSV, réduction des transferts réseau.
- Azure SQL Basic dimensionné au **strict minimum** du volume v1 (<500k lignes).

- Pipeline déclenché **à la demande** (pas de polling continu) pour éviter les traitements inutiles.
- Région *France Central* : datacentres Microsoft avec engagement 100 % énergies renouvelables.

5 Bilan opérationnel et financier

5.1 Réalisations – Indicateurs opérationnels

Indicateur	Cible	Réalisé	Statut
Communes couvertes (HdF)	≈3 700	3 709	Atteint
Indicateurs par commune	≥8	12	Atteint
Sources intégrées	≥4	5	Atteint
Valeurs manquantes (indicateurs clés)	<5 %	≈3 %	Atteint
Temps de réponse API (p95)	<2s	≈0.8s	Atteint
Disponibilité API	≥99 %	99.5 %	Atteint
Lignes SIRENE traitées (HdF actifs)	–	≈400k	Livré
Délai de livraison	4 mois	4 mois	Respecté

5.2 Bilan financier

Poste	Prévu	Réel	Écart
Azure SQL Database (4 mois)	20€	18€	-2€
Azure Data Lake Storage Gen2	40€	35€	-5€
Databricks (cluster spot)	60€	52€	-8€
Azure Key Vault + Monitor	28€	25€	-3€
Total infrastructure	148€	130€	-18€
Licences logicielles	0€	0€	=
Total projet	148€	130€	-12 %

Le projet a été livré **sous budget** grâce au recours aux clusters spot Databricks et à l'utilisation du tier gratuit Azure App Service (F1) pour l'hébergement de l'API.

5.3 Gouvernance et supervision

La supervision du pipeline et des services déployés est assurée via **Azure Monitor** :

- **Alertes** sur les transactions de stockage ADLS Gen2 (seuil : >500 transactions/heure).
- **Alertes** sur la capacité de stockage (seuil : >80 % de la capacité provisionnée).
- **Logs d'accès** à l'API FastAPI : journalisés dans Azure Application Insights.
- **Dashboard Azure Monitor** : visualisation en temps réel des métriques clés.

La **gouvernance des données** repose sur :

- Un **dictionnaire de données** maintenu dans le dépôt Git (`docs/architecture.md`).
- Des **tests de qualité** embarqués dans le pipeline (contrôle des types, des plages de valeurs, de la complétude).
- Un **RBAC** Azure granulaire : accès en lecture seule pour les analystes, accès complet pour le data engineer.

— Des **tags Azure** sur toutes les ressources : `projet=data-hdf`, `env=prod`, `owner=elbrek`.

6 Retour d'expérience – Problèmes et arbitrages

Problème rencontré	Description	Arbitrage / Solution
Secret stat. INSEE	Les communes <500 hab. n'ont pas de données FILOSOFI publiées (secret statistique).	Valeurs manquantes imputées par la médiane départementale ; flag <code>secret_stat=true</code> ajouté pour traçabilité.
Fusions de communes	Les codes INSEE évoluent lors de fusions (communes nouvelles). Certains codes du CSV INSEE ne correspondaient plus à l'API Géo.	Ajout d'une table de correspondance <code>dim_commune_historique</code> ; jointure sur le code INSEE le plus récent.
Volume SIRENE	Le fichier SIRENE brut dépasse 30M de lignes – pandas insuffisant (OOM).	Migration vers PySpark sur Databricks ; filtrage dès la lecture (pushdown) ; partitionnement par département.
Scraping Meilleurtaux	Structure HTML de la page modifiée en cours de projet, cassant le parser.	Basculement vers l'interception des requêtes AJAX (JSON natif), plus robuste que le parsing HTML.
Authentification Azure	Gestion des credentials Azure entre environnement local et CI/CD GitHub Actions.	Utilisation de <code>DefaultAzureCredential</code> (SDK Azure) + secrets GitHub Actions pour le CI/CD ; aucun mot de passe en dur.
Périmètre CRM	Demande en cours de projet d'intégrer les données clients internes (DCP).	Repoussé en v2 après AIPD ; la v1 reste 100 % données publiques non-personnelles.

Leçons apprises :

- Anticiper les limites de pandas dès la conception pour les sources volumineuses (>1M lignes).
- Documenter les codes INSEE historiques : les fusions de communes sont une source de bugs silencieux.
- Préférer les API officielles au scraping HTML quand elles existent – plus stables et fiables.
- Impliquer l'équipe conformité dès le cadrage, pas en fin de projet.

7 Bulletin de veille technique

Thème : Le format Apache Parquet et l'architecture Data Lakehouse

En quoi le format Parquet transforme-t-il le traitement des données massives, et pourquoi l'architecture Data Lakehouse s'impose-t-elle comme standard dans les projets data modernes ?

7.1 Qu'est-ce que le format Apache Parquet ?

Apache Parquet est un format de fichier **colonnaire** open-source conçu pour le traitement analytique de grands volumes de données. Contrairement aux formats ligne par ligne (CSV, JSON), Parquet organise les données colonne par colonne, ce qui permet :

- De ne lire que les colonnes nécessaires à une requête (**projection pushdown**) – gain I/O majeur.
- D'appliquer une **compression efficace** par colonne (Snappy, Gzip, ZSTD) : compression $\times 5$ à $\times 10$ vs CSV.
- D'intégrer nativement le **schéma des données** (typage fort), évitant les erreurs d'inférence.
- D'être lu nativement par l'ensemble des moteurs data modernes : Spark, DuckDB, Pandas, Polars, BigQuery, Redshift.

Application dans ce projet : Le fichier SIRENE brut de 30+ millions de lignes (environ 4 Go en CSV) est réduit à $\approx 400k$ lignes après filtrage HdF et stocké en Parquet, passant à environ 80 Mo – soit une réduction de 98 % du volume, avec un temps de lecture Spark divisé par 8.

7.2 L'architecture Data Lakehouse

Le **Data Lakehouse** est une architecture hybride combinant les avantages du **Data Lake** (stockage brut, pas de schéma imposé, coût faible) et du **Data Warehouse** (requêtes SQL rapides, données structurées, transactions ACID).

	Data Lake	Data Lakehouse
Format	Brut (CSV, JSON, images)	Parquet + Delta Lake / Iceberg
Requêtes SQL	Limitées	Performantes (moteur SQL direct)
Transactions ACID	Non	Oui
Gouvernance	Faible	Forte (catalog, RBAC)
Coût stockage	Faible	Faible

Ce projet implémente une architecture **Lakehouse simplifiée** sur Azure avec les zones **raw/** (données brutes), **staging/** (données nettoyées) et **curated/** (données prêtes à l'emploi) dans Azure Data Lake Storage Gen2, alimentant ensuite Azure SQL Database pour les requêtes analytiques finales.

7.3 Tendances et recommandations

Tendances observées (2024-2025) :

- **Delta Lake** (Databricks) et **Apache Iceberg** (open standard) ajoutent les transactions

ACID et le voyage dans le temps (*time travel*) sur les fichiers Parquet – adoption croissante en production.

- **DuckDB** permet d'interroger directement des fichiers Parquet avec SQL depuis un ordinateur portable, sans cluster – idéal pour les projets data de taille intermédiaire.
- **Polars** (Python) remplace progressivement pandas pour les traitements Parquet : 5 à 50× plus rapide grâce à son moteur Rust et son évaluation lazy.

Recommandations pour la v2 du projet :

- Migrer les zones **curated/** vers **Delta Lake** pour bénéficier des transactions ACID et de la gestion des mises à jour incrémentales (évite de re-traiter tout le fichier SIRENE chaque mois).
- Adopter **Microsoft Purview** comme catalogue de données pour documenter et gouverner l'ensemble des assets de données (sources, tables, colonnes, propriétaires, classifications RGPD).
- Mettre en place un **pipeline CI/CD** complet sur GitHub Actions pour automatiser les tests de qualité des données (Great Expectations) à chaque ingestion.